

Linked List Operations

Contents

1	Introduction	2
2	What is Linked List?	2
3	Types of Linked Lists	2
4	Linked List Operations	3
4.1	Insertion in Linked List	3
4.2	Deletion in Linked List	6
4.3	Traverse a Linked List	9
4.4	Searching for Element in Linked List	9
4.5	Reverse a Linked List	10
4.6	Sort a Linked List	10
4.7	Calculate Length of Linked List	11
5	Source Code	12
5.1	Linked List in Java	12
5.2	Linked List in Python	16
5.3	Linked List in C	19
5.4	Linked List in C++	23

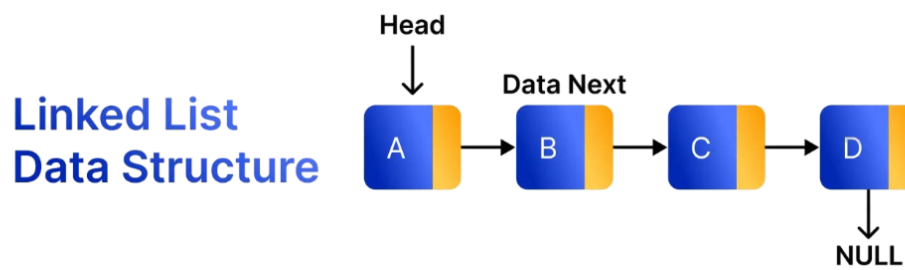
1 Introduction

Linked lists are a fundamental data structure in computer science, used to store a collection of elements. Unlike **arrays**, linked lists allow for efficient insertions and deletions, making them ideal for dynamic data management.

Let's learn about various linked list operations in data structure, including insertion, deletion, traversal, reversal, sorting, and cycle detection, with detailed explanations and example code.

2 What is Linked List?

A **linked list** is a linear data structure where elements, called nodes, are stored in non-contiguous memory locations. Each node contains a data part and a reference (or pointer) to the next node in the sequence.



3 Types of Linked Lists

1. Singly Linked List

- Each node points to the next node.
- Traversal is unidirectional (forward only).

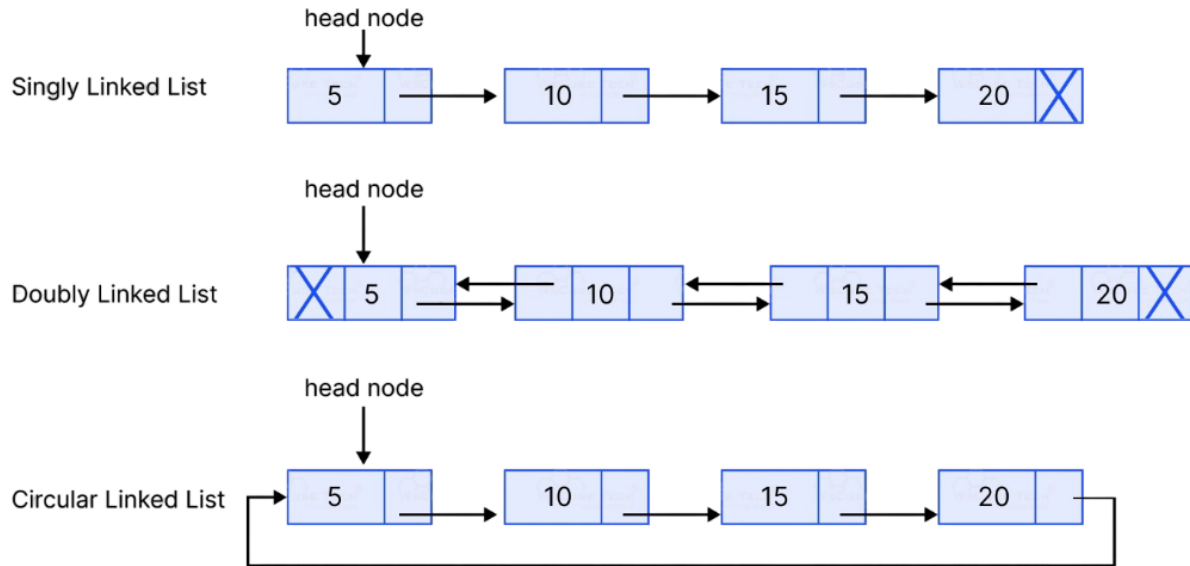
2. Doubly Linked List

- Each node points to both the next and previous nodes.
- Allows bidirectional traversal (forward and backward).

3. Circular Linked List

- The last node points back to the first node.
- Can be singly or doubly linked.
- Forms a circular structure, enabling continuous traversal.

Types of linked list



4 Linked List Operations

Let's know about the operations on linked lists in data structure with examples.

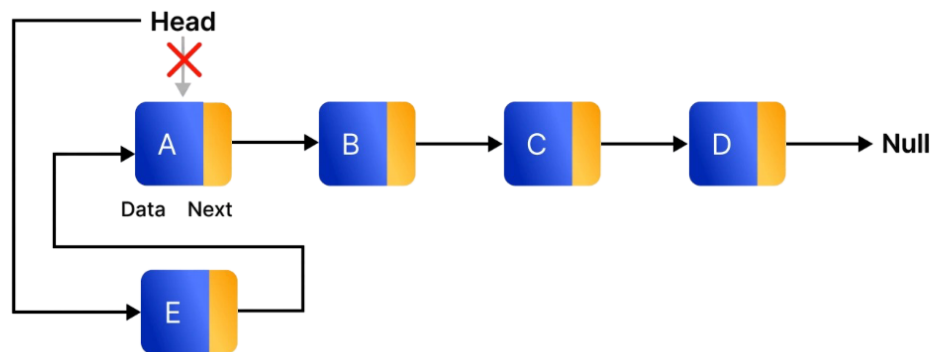
4.1 Insertion in Linked List

Linked list insertion operations can be performed at the beginning, end, and specific positions.

- **Insertion at the Beginning**

Steps:

- Create a new node.
- Set the new node's next pointer to the current head.
- Update the head to point to the new node.



Example:



```

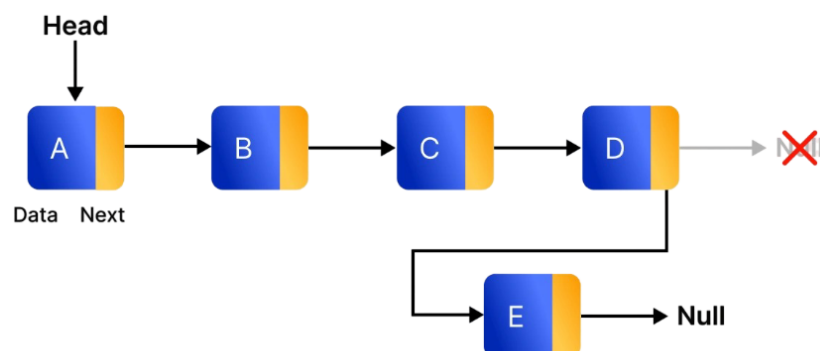
1 void insertAtBeginning(int data) {
2     Node newNode = new Node(data);
3     newNode.next = head;
4     head = newNode;
5 }

```

• Insertion at the End

Steps:

- Create a new node.
- Traverse to the end of the list.
- Set the last node's next pointer to the new node.



Example:



```

1 void insertAtEnd(int data) {
    Node newNode = new Node(data);
3   if (head == null) {
        head = newNode;
5       return;
    }

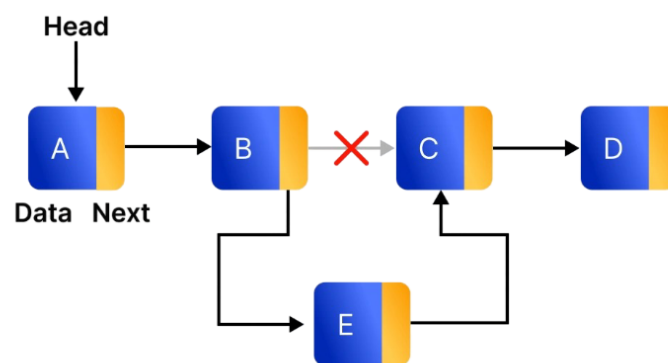
7
    Node last = head;
9   while (last.next != null) {
        last = last.next;
11  }
    last.next = newNode;
13 }

```

• Insertion at a Specific Position

Steps:

- Create a new node.
- Traverse to the node before the desired position.
- Set the new node's next pointer to the current node's next.
- Update the current node's next pointer to the new node.



Example:



```

1 void insertAtPosition(int data, int position) {
    Node newNode = new Node(data);
3   if (position == 0) {

```



```

5     newNode.next = head;
    head = newNode;
    return;
7 }

9     Node current = head;
    for (int i = 0; i < position - 1; i++) {
11         if (current == null) {
            throw new IndexOutOfBoundsException("Position out of range");
13         }
            current = current.next;
15     }
    newNode.next = current.next;
17     current.next = newNode;
    }

```

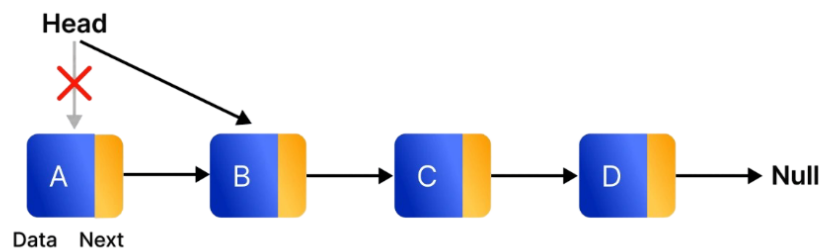
4.2 Deletion in Linked List

We will discuss how to delete a node from linked list from the beginning, end, and specific positions:

- Deletion from the Beginning

Steps:

- Check if the list is empty.
- Set the head to the next node.



Example:



```

void deleteFromBeginning() {
2     if (head == null) {
        return;
4     }

```

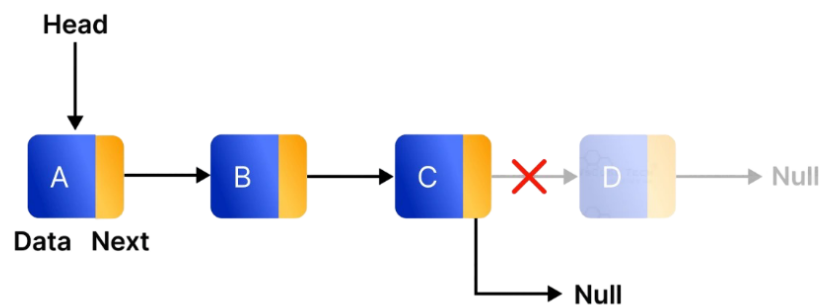


```
6  head = head.next;
   }
```

• Deletion from the End

Steps:

- Check if the list is empty.
- Traverse to the second-to-last node.
- Set the second-to-last node's next pointer to None.



Example:



```
void deleteFromEnd() {
2  if (head == null) {
    return;
4  }

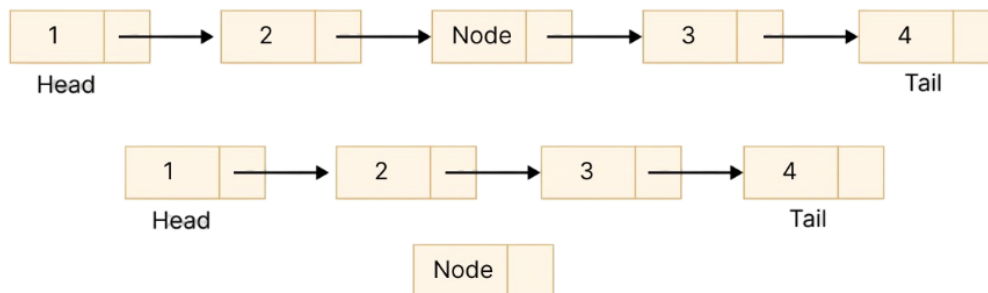
6  if (head.next == null) {
    head = null;
8    return;
  }

10 Node secondLast = head;
12 while (secondLast.next.next != null) {
    secondLast = secondLast.next;
14 }
    secondLast.next = null;
16 }
```

• Deletion from a Specific Position

Steps:

- Check if the list is empty.
- Traverse to the node before the desired position.
- Update the node's next pointer to skip the node at the desired position.



Example:



```

1 void deleteFromPosition(int position) {
2     if (head == null) {
3         return;
4     }
5
6     if (position == 0) {
7         head = head.next;
8         return;
9     }
10
11    Node current = head;
12    for (int i = 0; i < position - 1; i++) {
13        if (current.next == null) {
14            throw new IndexOutOfBoundsException("Position out of range");
15        }
16        current = current.next;
17    }
18    if (current.next == null) {
19        throw new IndexOutOfBoundsException("Position out of range");
20    }
21    current.next = current.next.next;
22 }

```


4.3 Traverse a Linked List

Now, let's learn how to perform linked list traversal and print elements.

Steps:

- Start from the head.
- Traverse through each node and print its data.

Example:



```
1 void traverse() {  
    Node current = head;  
3 while (current != null) {  
    System.out.print(current.data + " -> ");  
5    current = current.next;  
    }  
7 System.out.println("None");  
}
```

4.4 Searching for Element in Linked List

Here, we will discuss how to perform search operations in linked list:

Steps:

- Start from the head.
- Traverse through each node and compare its data with the target value.

Example:



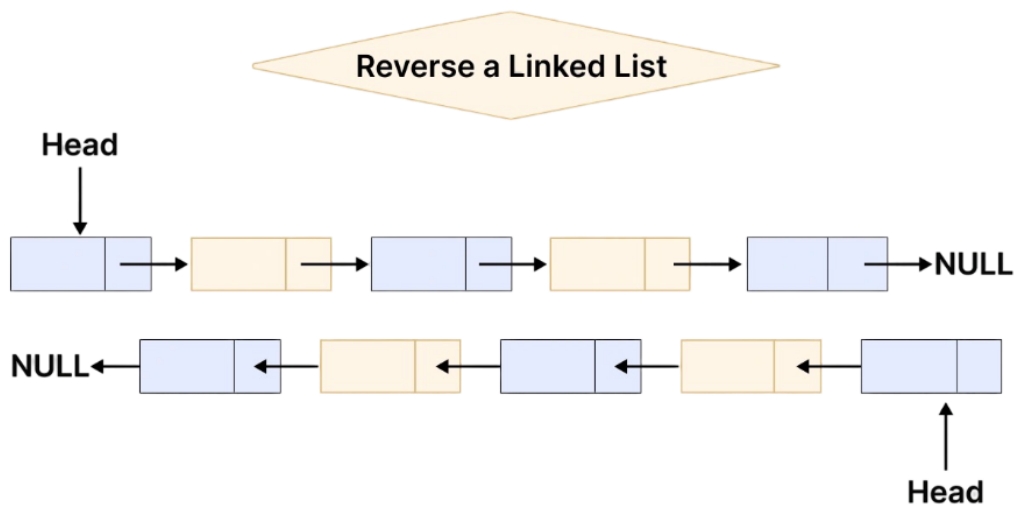
```
boolean search(int target) {  
2 Node current = head;  
    while (current != null) {  
4     if (current.data == target) {  
        return true;  
6     }  
        current = current.next;  
8     }  
    return false;  
10 }
```

4.5 Reverse a Linked List

Next linked list operation in data structures is reverse.

Steps:

- Initialize three pointers: prev, current, and next.
- Traverse the list and reverse the pointers.



Example:



```

1 void reverse() {
2     Node prev = null;
3     Node current = head;
4     Node next;
5     while (current != null) {
6         next = current.next;
7         current.next = prev;
8         prev = current;
9         current = next;
10    }
11    head = prev;
12 }

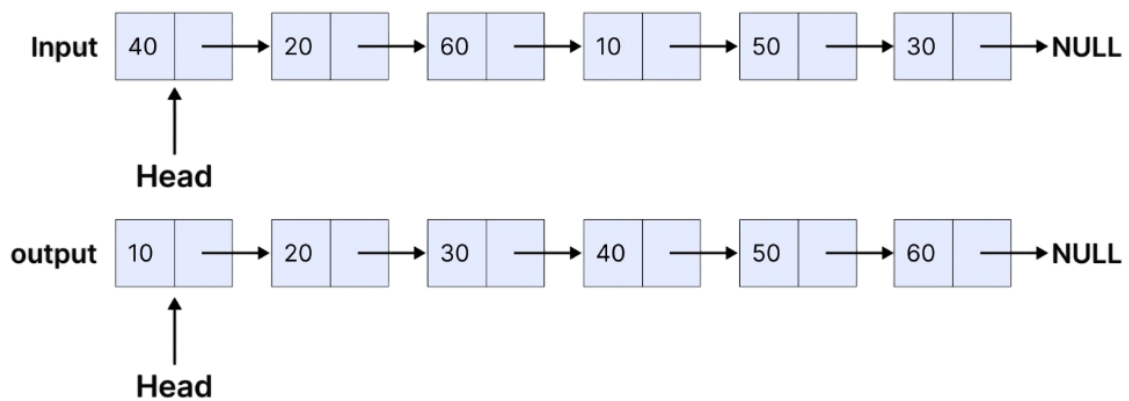
```

4.6 Sort a Linked List

Next in the list of linked list operations is sorting the elements:

Steps:

- Extract elements into an array.
- Sort the array.
- Rebuild the linked list from the sorted array.



Example:



```

1 void sort() {
2     if (head == null || head.next == null) {
3         return;
4     }
5
6     Node current = head;
7     while (current != null) {
8         Node next = current.next;
9         while (next != null) {
10            if (current.data > next.data) {
11                int temp = current.data;
12                current.data = next.data;
13                next.data = temp;
14            }
15            next = next.next;
16        }
17        current = current.next;
18    }
19 }

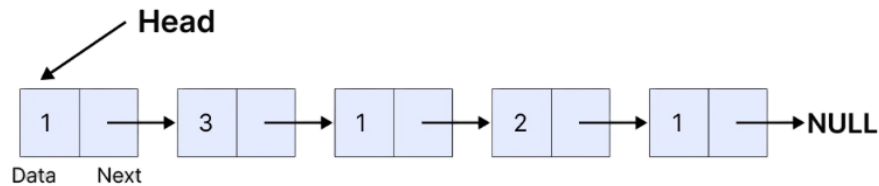
```

4.7 Calculate Length of Linked List

Lastly, we will discuss how to perform linked list length calculation.

Steps:

- Initialize a counter.
- Traverse through each node and increment the counter.



Example:



```
1 int length() {
    int count = 0;
3   Node current = head;
    while (current != null) {
5       count++;
        current = current.next;
7   }
    return count;
9 }
```

5 Source Code

5.1 Linked List in Java



```
1 class Node {
    int data;
3   Node next;

5   Node(int data) {
        this.data = data;
7       this.next = null;
    }
9 }

11 class LinkedList {
    Node head;
13
    LinkedList() {
15     this.head = null;
    }
```



```
17 void insertAtBeginning(int data) {
19     Node newNode = new Node(data);
20     newNode.next = head;
21     head = newNode;
22 }
23
24 void insertAtEnd(int data) {
25     Node newNode = new Node(data);
26     if (head == null) {
27         head = newNode;
28         return;
29     }
30     Node last = head;
31     while (last.next != null) {
32         last = last.next;
33     }
34     last.next = newNode;
35 }
36
37 void insertAtPosition(int data, int position) {
38     Node newNode = new Node(data);
39     if (position == 0) {
40         newNode.next = head;
41         head = newNode;
42         return;
43     }
44     Node current = head;
45     for (int i = 0; i < position - 1; i++) {
46         if (current == null) {
47             throw new IndexOutOfBoundsException("Position out of range");
48         }
49         current = current.next;
50     }
51     newNode.next = current.next;
52     current.next = newNode;
53 }
54
55 void deleteFromBeginning() {
56     if (head == null) {
57         return;
58     }
59     head = head.next;
60 }
61
62 void deleteFromEnd() {
63     if (head == null) {
64         return;
65     }
66     if (head.next == null) {
67         head = null;
68         return;
69     }
70 }
```



```
69     }
70     Node secondLast = head;
71     while (secondLast.next.next != null) {
72         secondLast = secondLast.next;
73     }
74     secondLast.next = null;
75 }
76
77 void deleteFromPosition(int position) {
78     if (head == null) {
79         return;
80     }
81     if (position == 0) {
82         head = head.next;
83         return;
84     }
85     Node current = head;
86     for (int i = 0; i < position - 1; i++) {
87         if (current.next == null) {
88             throw new IndexOutOfBoundsException("Position out of range");
89         }
90         current = current.next;
91     }
92     if (current.next == null) {
93         throw new IndexOutOfBoundsException("Position out of range");
94     }
95     current.next = current.next.next;
96 }
97
98 void traverse() {
99     Node current = head;
100    while (current != null) {
101        System.out.print(current.data + " -> ");
102        current = current.next;
103    }
104    System.out.println("None");
105 }
106
107 boolean search(int target) {
108     Node current = head;
109     while (current != null) {
110         if (current.data == target) {
111             return true;
112         }
113         current = current.next;
114     }
115     return false;
116 }
117
118 void reverse() {
119     Node prev = null;
120     Node current = head;
```



```
121     Node next;
122     while (current != null) {
123         next = current.next;
124         current.next = prev;
125         prev = current;
126         current = next;
127     }
128     head = prev;
129 }
130
131 void sort() {
132     if (head == null || head.next == null) {
133         return;
134     }
135     Node current = head;
136     while (current != null) {
137         Node next = current.next;
138         while (next != null) {
139             if (current.data > next.data) {
140                 int temp = current.data;
141                 current.data = next.data;
142                 next.data = temp;
143             }
144             next = next.next;
145         }
146         current = current.next;
147     }
148 }
149
150 int length() {
151     int count = 0;
152     Node current = head;
153     while (current != null) {
154         count++;
155         current = current.next;
156     }
157     return count;
158 }
159
160 public static void main(String[] args) {
161     LinkedList ll = new LinkedList();
162     ll.insertAtEnd(10);
163     ll.insertAtEnd(20);
164     ll.insertAtEnd(30);
165     ll.traverse();
166
167     ll.insertAtBeginning(5);
168     ll.traverse();
169
170     ll.insertAtPosition(25, 2);
171     ll.traverse();
```



```

173     ll.deleteFromBeginning();
174     ll.traverse();
175
176     ll.deleteFromEnd();
177     ll.traverse();
178
179     ll.deleteFromPosition(1);
180     ll.traverse();
181
182     System.out.println("List contains 20: " + ll.search(20));
183     System.out.println("List contains 40: " + ll.search(40));
184
185     ll.reverse();
186     ll.traverse();
187
188     ll.insertAtEnd(15);
189     ll.insertAtEnd(5);
190     ll.traverse();
191
192     ll.sort();
193     ll.traverse();
194
195     System.out.println("Length of the list: " + ll.length());
196 }
197 }

```

5.2 Linked List in Python



```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9
10    def insert_at_beginning(self, data):
11        new_node = Node(data)
12        new_node.next = self.head
13        self.head = new_node
14
15    def insert_at_end(self, data):
16        new_node = Node(data)
17        if self.head is None:
18            self.head = new_node
19        return

```




```
last = self.head
21 while last.next:
    last = last.next
23 last.next = new_node

25 def insert_at_position(self, data, position):
    new_node = Node(data)
    27 if position == 0:
        new_node.next = self.head
    29 self.head = new_node
        return
    31 current = self.head
    for i in range(position - 1):
    33     if current is None:
        raise IndexError("Position out of range")
    35     current = current.next
    new_node.next = current.next
    37 current.next = new_node

39 def delete_from_beginning(self):
    if self.head is None:
    41     return
    self.head = self.head.next
    43

45 def delete_from_end(self):
    if self.head is None:
        return
    47 if self.head.next is None:
        self.head = None
        return
    49 second_last = self.head
    while second_last.next.next:
    51     second_last = second_last.next
    53 second_last.next = None

55 def delete_from_position(self, position):
    if self.head is None:
    57     return
    if position == 0:
    59     self.head = self.head.next
        return
    61 current = self.head
    for i in range(position - 1):
    63     if current.next is None:
        raise IndexError("Position out of range")
    65     current = current.next
    if current.next is None:
    67     raise IndexError("Position out of range")
    current.next = current.next.next
    69

71 def traverse(self):
    current = self.head
```



```
while current:
    print(current.data, end=" -> ")
    current = current.next
    print("None")

def search(self, target):
    current = self.head
    while current:
        if current.data == target:
            return True
        current = current.next
    return False

def reverse(self):
    prev = None
    current = self.head
    while current:
        next = current.next
        current.next = prev
        prev = current
        current = next
    self.head = prev

def sort(self):
    if self.head is None or self.head.next is None:
        return
    elements = []
    current = self.head
    while current:
        elements.append(current.data)
        current = current.next
    elements.sort()
    self.head = Node(elements[0])
    current = self.head
    for data in elements[1:]:
        current.next = Node(data)
        current = current.next

def length(self):
    count = 0
    current = self.head
    while current:
        count += 1
        current = current.next
    return count
```

5.3 Lined List in C



```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int data) {
    struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtBeginning(struct Node **head, int data) {
    struct Node *newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

void insertAtEnd(struct Node **head, int data) {
    struct Node *newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void insertAtPosition(struct Node **head, int data, int position) {
    struct Node *newNode = createNode(data);
    if (position == 0) {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range\n");
    }
}
```



```
50     return;
51 }
52 newNode->next = temp->next;
53 temp->next = newNode;
54 }

56 void deleteFromBeginning(struct Node **head) {
57     if (*head == NULL) return;
58     struct Node *temp = *head;
59     *head = (*head)->next;
60     free(temp);
61 }

62 void deleteFromEnd(struct Node **head) {
63     if (*head == NULL) return;
64     if ((*head)->next == NULL) {
65         free(*head);
66         *head = NULL;
67         return;
68     }
69
70     struct Node *temp = *head;
71     while (temp->next->next != NULL) {
72         temp = temp->next;
73     }
74     free(temp->next);
75     temp->next = NULL;
76 }

78 void deleteFromPosition(struct Node **head, int position) {
79     if (*head == NULL) return;
80     struct Node *temp = *head;
81     if (position == 0) {
82         *head = temp->next;
83         free(temp);
84         return;
85     }
86
87     for (int i = 0; temp != NULL && i < position - 1; i++) {
88         temp = temp->next;
89     }
90     if (temp == NULL || temp->next == NULL) {
91         printf("Position out of range\n");
92         return;
93     }
94     struct Node *next = temp->next->next;
95     free(temp->next);
96     temp->next = next;
97 }

98 }

100 void traverse(struct Node *head) {
101     struct Node *temp = head;
```



```

102 while (temp != NULL) {
103     printf("%d -> ", temp->data);
104     temp = temp->next;
105 }
106 printf("NULL\n");
107 }
108
109 int search(struct Node *head, int target) {
110     struct Node *temp = head;
111     while (temp != NULL) {
112         if (temp->data == target) return 1;
113         temp = temp->next;
114     }
115     return 0;
116 }
117
118 void reverse(struct Node **head) {
119     struct Node *prev = NULL;
120     struct Node *current = *head;
121     struct Node *next = NULL;
122     while (current != NULL) {
123         next = current->next;
124         current->next = prev;
125         prev = current;
126         current = next;
127     }
128     *head = prev;
129 }
130
131 void sort(struct Node **head) {
132     if (*head == NULL || (*head)->next == NULL) return;
133     struct Node *temp = *head;
134     while (temp != NULL) {
135         struct Node *next = temp->next;
136         while (next != NULL) {
137             if (temp->data > next->data) {
138                 int data = temp->data;
139                 temp->data = next->data;
140                 next->data = data;
141             }
142             next = next->next;
143         }
144         temp = temp->next;
145     }
146 }
147
148 int length(struct Node *head) {
149     int count = 0;
150     struct Node *temp = head;
151     while (temp != NULL) {
152         count++;
153         temp = temp->next;
154     }
155 }

```



```
154 }
    return count;
156 }

158 int main() {
    struct Node *head = NULL;

160     insertAtEnd(&head, 10);
162     insertAtEnd(&head, 20);
164     insertAtEnd(&head, 30);
166     traverse(head);

168     insertAtBeginning(&head, 5);
170     traverse(head);

172     insertAtPosition(&head, 25, 2);
174     traverse(head);

176     deleteFromBeginning(&head);
178     traverse(head);

180     deleteFromEnd(&head);
182     traverse(head);

184     deleteFromPosition(&head, 1);
186     traverse(head);

188     printf("List contains 20: %s\n", search(head, 20) ? "Yes" : "No");
190     printf("List contains 40: %s\n", search(head, 40) ? "Yes" : "No");

192     reverse(&head);
194     traverse(head);

196     insertAtEnd(&head, 15);
198     insertAtEnd(&head, 5);
200     traverse(head);

202     sort(&head);
204     traverse(head);

206     printf("Length of the list: %d\n", length(head));

208     return 0;
210 }
```

5.4 Linked List in C++



```

1 #include <iostream>
  using namespace std;
3
  struct Node {
5     int data;
    Node *next;
7 };

9 Node *createNode(int data) {
    Node *newNode = new Node();
11  newNode->data = data;
    newNode->next = nullptr;
13  return newNode;
    }

15
  void insertAtBeginning(Node *&head, int data) {
17  Node *newNode = createNode(data);
    newNode->next = head;
19  head = newNode;
    }

21
  void insertAtEnd(Node *&head, int data) {
23  Node *newNode = createNode(data);
    if (head == nullptr) {
25      head = newNode;
      return;
27  }

29  Node *temp = head;
    while (temp->next != nullptr) {
31      temp = temp->next;
    }
33  temp->next = newNode;
    }

35
  void insertAtPosition(Node *&head, int data, int position) {
37  Node *newNode = createNode(data);
    if (position == 0) {
39      newNode->next = head;
      head = newNode;
41      return;
    }

43
    Node *temp = head;
45  for (int i = 0; temp != nullptr && i < position - 1; i++) {
        temp = temp->next;
47  }
    if (temp == nullptr) {
49      cout << "Position out of range" << endl;

```



```

    return;
51 }

53 newNode->next = temp->next;
    temp->next = newNode;
55 }

57 void deleteFromBeginning(Node *&head) {
    if (head == nullptr) return;
59 Node *temp = head;
    head = head->next;
61 delete temp;
    }
63

    void deleteFromEnd(Node *&head) {
65     if (head == nullptr) return;
        if (head->next == nullptr) {
67         delete head;
            head = nullptr;
69         return;
        }
71

        Node *temp = head;
73         while (temp->next->next != nullptr) {
            temp = temp->next;
75         }
            delete temp->next;
77         temp->next = nullptr;
        }
79

    void deleteFromPosition(Node *&head, int position) {
81     if (head == nullptr) return;
        Node *temp = head;
83     if (position == 0) {
            head = temp->next;
85         delete temp;
            return;
87     }

89     for (int i = 0; temp != nullptr && i < position - 1; i++) {
        temp = temp->next;
91     }
        if (temp == nullptr || temp->next == nullptr) {
93         cout << "Position out of range" << endl;
            return;
95         }

97     Node *next = temp->next->next;
        delete temp->next;
99     temp->next = next;
    }
101

```




```

void traverse(Node *head) {
103     Node *temp = head;
    while (temp != nullptr) {
105         cout << temp->data << " -> ";
        temp = temp->next;
107     }
    cout << "NULL" << endl;
109 }

111 bool search(Node *head, int target) {
    Node *temp = head;
113     while (temp != nullptr) {
        if (temp->data == target) return true;
115         temp = temp->next;
    }
117     return false;
}

119 void reverse(Node *&head) {
121     Node *prev = nullptr;
    Node *current = head;
123     Node *next = nullptr;
    while (current != nullptr) {
125         next = current->next;
        current->next = prev;
127         prev = current;
        current = next;
129     }
    head = prev;
131 }

133 void sort(Node *&head) {
    if (head == nullptr || head->next == nullptr) return;
135     Node *temp = head;
    while (temp != nullptr) {
137         Node *next = temp->next;
        while (next != nullptr) {
139             if (temp->data > next->data) {
                int data = temp->data;
                temp->data = next->data;
                next->data = data;
143             }
            next = next->next;
145         }
        temp = temp->next;
147     }
}

149 int length(Node *head) {
151     int count = 0;
    Node *temp = head;
153     while (temp != nullptr) {

```



```

155     count++;
        temp = temp->next;
    }
157     return count;
    }
159
161     int main() {
        Node *head = nullptr;

163         insertAtEnd(head, 10);
        insertAtEnd(head, 20);
165         insertAtEnd(head, 30);
        traverse(head);

167         insertAtBeginning(head, 5);
        traverse(head);

171         insertAtPosition(head, 25, 2);
        traverse(head);

173         deleteFromBeginning(head);
        traverse(head);

175         deleteFromEnd(head);
        traverse(head);

177         deleteFromPosition(head, 1);
        traverse(head);

181         cout << "List contains 20: " << (search(head, 20) ? "Yes" : "No") <<
            << endl;
        cout << "List contains 40: " << (search(head, 40) ? "Yes" : "No") <<
            << endl;

185         reverse(head);
        traverse(head);

187         insertAtEnd(head, 15);
        insertAtEnd(head, 5);
191         traverse(head);

193         sort(head);
        traverse(head);

195         cout << "Length of the list: " << length(head) << endl;

197         return 0;
199     }

```